Weekly Report - Practical Week 06

- **Project:** Personal Finance Manager
- **Group ID:** Group07
- **Report Period:** 2026-11-07 to 2026-11-13

1. Summary of Weekly Activities

> Add content here

2. Individual Member Reports

Huỳnh Trần Gia Hân (`htghan-dotcom`)

- **Assigned Module:** Wallet Management & Data Persistence.
- **Tasks Planned for the Week:**
  - Set up the CMake build configuration and build documentation for the project.
  - Implement the `Wallet` data structure and core CRUD operations using manual dynamic memory allocation.
- **Work Completed / Progress:**
  - Configured `CMakeLists.txt` (enforcing C++17 standard, header/source auto-discovery, and compiler warning flags) and updated the build instructions in `README.md`.
  - Defined the `Wallet` struct and implemented dynamic array management functions (`addWallet`, `editWallet`,

`deleteWallet`, `freeWallets`) without using STL containers.
    - Implemented lookup and balance helper utilities (`findWalletIndexById`, `getWalletById`, `updateWalletBalance`, `getTotalBalance`) to support transaction processing and dashboard summaries.
- **Obstacles / Blockers:** None.
- **Next Week Plan:**
    - Implement binary file I/O (`saveWalletsToBinary` and `loadWalletsFromBinary`) for session persistence.
    - Integrate `Wallet` operations with the Transaction module and main CLI Dashboard.

Nguyễn Gia Khánh (`gkhanhhh-debug`)

- **Assigned Module:** Master Data & CLI Navigation Framework
- **Tasks Planned for the Week:**
    - Complete the remaining CRUD methods in `MasterDataManager` (update, delete, and validation).
    - Scaffold the CLI navigation loop and primary Dashboard screen.
- **Work Completed / Progress:**
    - Refactored `MasterDataManager` to remove `std::vector` and manage `IncomeSource` and `ExpenseCategory` using manual dynamic arrays (`new[]`, `delete[]`, custom `resize()`, and index-shifting deletion).
    - Completed all 8 CRUD methods (`add`, formatted table `display` using `<iomanip>`, `update`, and `delete`) with input buffer validation.
    - Added `namespace ConsoleUtils` (`clearScreen`, `pauseScreen`, `getValidChoice`) into `Utils.cpp` to handle cross-platform screen clearing and safe menu input.
    - Implemented `DashboardView` (`renderDashboard`) synchronized with `Wallet.cpp` (`getTotalBalance`, `id`, `name`, `balance`).
- **Obstacles / Blockers:**
    - Refactoring existing STL-based logic into manual dynamic pointer arrays and aligning struct field names with

`Wallet.cpp` delayed the final implementation of `MenuSystem`.

- **Next Week Plan:**
  - Implement `MenuSystem` (multi-level CLI navigation loop and automated routing flow).
  - Integrate `MasterDataManager`, `Wallet`, `Transaction`, and `Utils` inside `main.cpp` for end-to-end testing.

Nguyễn Bá Đạt (`DatJust`)

- **Assigned Module:** Transaction Management & Shared Utilities
- **Tasks Planned for the Week:**
  - Scaffold `IncomeTransaction` and `ExpenseTransaction` structs and integrate them with the Date utility.
  - Implement basic balance update logic on transaction creation.
- **Work Completed / Progress:**
  - Implemented dynamic array management for income transactions in `Transaction.cpp` (`nextIncomeId`, `recordIncome`, `incomeToString`, and `freeIncomeList`).
  - Added dynamic capacity doubling and validation checks for positive amounts, non-empty dates, and valid IDs.
  - Maintained Date parsing and validation helpers in `Utils.cpp`.
- **Obstacles / Blockers:**
  - Awaiting `MenuSystem` integration to wire foreign key checks with `MasterDataManager` and automatic balance updates via `updateWalletBalance()`.
- **Next Week Plan:**
  - Implement `ExpenseTransaction` dynamic array CRUD operations (`recordExpense`, `expenseToString`, `freeExpenseList`).
  - Connect transaction recording with `Utils` date validation and `Wallet` balance updates.

Nguyễn Đức Thành Đạt (`hiepsimauhong`)

- **Assigned Module:** Recurring Automation & Financial Statistics
- **Tasks Planned for the Week:**
  - Implement `RecurringTransaction` data models and define the monthly trigger cycle algorithm.
  - Begin algorithmic design for time-based net balance calculations.
- **Work Completed / Progress:**
  - No code commits or module implementations were completed during this reporting window due to university midterm examinations.
- **Obstacles / Blockers:**
  - Ongoing university midterm examination schedule prevented active development during Practical Week 06.
- **Next Week Plan:**
  - Implement `RecurringTransaction` data models and define the monthly trigger cycle algorithm using `Utils.cpp` date helpers.
  - Implement time-based financial summary and net balance calculation functions.

---

3. Sprint Retrospective & Overall Project Status

- **Repository Status:** Core dynamic array modules (`MasterDataManager`, `Wallet`, `Transaction` income functions, `Utils` with `ConsoleUtils`, and `DashboardView`) are implemented without STL containers.
- **Timeline Assessment:** Slightly adjusted due to the refactoring from `std::vector` to raw dynamic arrays and Thành Đạt's midterm exam schedule. Priority for Practical Week 07 is completing `MenuSystem`, `ExpenseTransaction`, binary file persistence, and `RecurringTransaction` modules for full integration